

Module 4 In-class Coding Submission I



In-class Coding Submission: Java Exception Control Examples

Objective: The objective of this lab assignment is to create three Java programs demonstrating the use of exception handling to control and manage errors.

Instructions:

Program 1: Checked Exception Handling

1. Objective:

- Demonstrate the handling of a checked exception.

2. Requirements:

- Create a Java program that reads data from a file.
- Handle the `FileNotFoundException` using the appropriate exception-handling mechanism.
- Provide meaningful messages in the exception handling block.

3. Program Execution:

- The program should run without errors.
- Include sample input and output to showcase the exception handling.
- Use the following file for file reading: [SalesData.txt \(https://uciunex.instructure.com/courses/21820/files/3143028?wrap=1\)](https://uciunex.instructure.com/courses/21820/files/3143028?wrap=1) 
 [\(https://uciunex.instructure.com/courses/21820/files/3143028/download?download_frd=1\)](https://uciunex.instructure.com/courses/21820/files/3143028/download?download_frd=1) 

4. Skeleton Code:

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class FileReadExample {
    public static void main(String[] args) {
        try {
            // Your code here
        } catch (FileNotFoundException e) {
            // Handle the exception
        }
    }
}
```

Program 2: Unchecked Exception Handling

1. Objective:

- Demonstrate the handling of an unchecked exception.

2. Requirements:

- Create a Java program that performs a division operation.
- Handle the `ArithmeticException` that may occur during division by zero.
- Provide meaningful messages in the exception handling block.

3. Program Execution:

- The program should run without errors.
- Include sample input and output to showcase the exception handling.

4. Skeleton Code:

```
public class DivisionExample {  
    public static void main(String[] args) {  
        try {  
            // Your code here  
        } catch (ArithmeticException e) {  
            // Handle the exception  
        }  
    }  
}
```

Program 3: Custom Exception Handling

1. Objective:

- Create a custom exception and demonstrate its handling.

2. Requirements:

- Create a Java program that validates user input for a positive number.
- If the user enters a non-positive number, throw a custom exception (e.g., `NegativeNumberException`).
- Handle the custom exception and provide meaningful messages.

3. Program Execution:

- The program should run without errors.
- Include sample input and output to showcase the custom exception handling.

4. Skeleton Code:

```
class NegativeNumberException extends Exception {
    public NegativeNumberException(String message) {
        super(message);
    }
}

public class PositiveNumberValidator {
    public static void main(String[] args) {
        try {
            // Your code here
        } catch (NegativeNumberException e) {
            // Handle the exception
        }
    }

    private static void validatePositiveNumber(int number) throws NegativeNumberException {
        // Your code here
    }
}
```

Grading Rubric (Maximum 100 Points):

- **Program 1 (Checked Exception Handling): 30 Points**
 - Correct implementation of file reading: 10 points
 - Appropriate handling of `FileNotFoundException`: 10 points
 - Meaningful exception messages: 10 points
- **Program 2 (Unchecked Exception Handling): 30 Points**
 - Correct implementation of division operation: 10 points
 - Appropriate handling of `ArithmeticException`: 10 points
 - Meaningful exception messages: 10 points
- **Program 3 (Custom Exception Handling): 40 Points**
 - Implementation of custom exception (`NegativeNumberException`): 10 points
 - Correct validation of user input: 10 points
 - Proper handling of the custom exception: 10 points
 - Meaningful exception messages: 10 points

Submission: Submit the Java source code for each program along with a document describing the purpose of each program, the exception-handling approach used, and screenshots or console outputs showing the programs' successful execution. Ensure that your submission adheres to the given instructions and grading rubric.

Points 100

Submitting a file upload

Due	For	Available from	Until
Oct 1	Everyone	-	Oct 22 at 11:59pm